

No.	Questions
	What is Docker?
	Docker Installation?
	TO check version?
	Check the all docker Commands

	Create docker image
	Show docker images
	Delte the images
	Build an image from a Dockerfile

	Image to container
	Check container list
	container start
	Delete the container
	Run container with custom name
	container run in background/detached mode

What is Docker

- Docker is an open-source platform that allows developers to package, deploy, and run applications as containers. A container is a lightweight, portable environment that houses an application and all of its dependencies.
- The application can be run anywhere (Windows, Linux, Cloud) in the same way.* We can run code and dependencies in containers without installing them directly on your system.

Docker Installation

- search **get docker** on google (OR) hit "<https://docs.docker.com/get-started/get-docker/>"
- click on Docker desktop for window then click on **Docker Desktop for Windows - x86_64**

TO check version

```
# opne cmd
docker -v
```

Check the all docker Commands

```
# opne cmd
docker
```

Image Commands

Create docker image

- If this image is not present in our local environment then it is pulled from Docker Hub.

```
docker pull hello-world
docker pull mysql
docker pull mysql:8.0    #if any specific version or tag
```

Show docker images

```
docker images
```

Delte the images

- delete the container first related to image

```
docker rmi IMAGE_ID
docker rmi mohit2708/myfastapiapp:latest    #delete with specific name
docker rmi -f IMAGE_ID    #Force delete
docker image prune        #Remove unused images
```

Build an image from a Dockerfile

```
docker build -t <image_name>:<version> .          #version is optional
docker build -t <image_name>:<version> . -no-cache  #build without cache
```

What Are Docker Image Layers?

- A Docker image is made up of a series of layers stacked on top of each other.
- Each layer represents a change or addition made to the image, like:
 - installing software
 - copying files
 - setting environment variables
- These layers are read-only and are created during the build process, usually from a Dockerfile.

Why Layers Matter

- ☒ 1. **Reusability & Caching:** Docker caches layers, so if you rebuild the image and some layers haven't changed, it reuses them — making builds much faster.
 - For example: If you only changed files in your app folder, only the COPY . /app layer will be rebuilt.
- ☒ 2. **Efficiency:-** Multiple containers can share the same layers on disk, saving space.
- ☒ 3. **Transparency** Each layer has a unique ID and can be inspected (docker history image_name) to see what changes were made at each step.

Container commands

Run the container

☒ Create & run a new container/Image to container

- if image not available locally, it'll be downloaded from DockerHub

```
docker run image_id/image_name
```

Run container using Port Binding

- The first 8000 (before the colon) is the **host port** — the port on your computer.
- The second 8000 (after the colon) is the **container port** — the port inside the Docker container where your app is running.

```
docker run -p<host_port>:<container_port> <image_name>  
docker run -d -p 8000:8000 myfastapiapp // example
```

Run container with custom name

```
docker run --name <container_name> <image_name>
```

container run in background/detached mode

- `docker run -d image_name` में `-d` का मतलब है detached mode।
- `-d` (detached mode) का मतलब है कि कंटेनर background में चलेगा।
- आप कंटेनर को स्टार्ट करेंगे, लेकिन उसका टर्मिनल या आउटपुट आपके स्क्रीन पर नहीं दिखेगा।
- कंटेनर अपने आप काम करता रहेगा और आप अपना टर्मिनल फ्री रख सकते हैं।

```
docker run -d <image_name>
```

उदाहरण:

```
docker run -d nginx
```

- ये कमांड Nginx वेब सर्वर कंटेनर को बैकग्राउंड में रन करता है।
- आप कंटेनर के लॉग देखने के लिए बाद में `docker logs <container_id>` चला सकते हैं।

- या कंटेनर को मैनेज करने के लिए `docker ps` से कंटेनर की लिस्ट देख सकते हैं।
- जब आपको कंटेनर को लगातार बैकग्राउंड में चलाना हो, जैसे कि वेब सर्वर, डेटाबेस, या कोई सर्विस।
- जब आपको कंटेनर के साथ इंटरैक्ट करने की जरूरत न हो, बल्कि वह अपने आप काम करता रहे।

Example

- In Docker, the **-e** option is used to set environment variables inside the container at runtime.

```
docker run -d --name my-mysql -e MYSQL_ROOT_PASSWORD=my-secret-pw mysql:latest
```

Set environment variables in a container

```
docker run -e <var_name>=<var_value> <container_name> (or <container_id>)
```

Check container list

```
# all Running containers
docker ps
# see all containers (running or stopped):
docker ps -a
```

container start/Stoped

```
docker start|stop container_id/container_name
```

Delete the container

```
-- Step 1:- stop the container
docker stop <container_id_or_name>
-- Step 2:- delete the container
docker rm <container_id_or_name>
(OR)-- stop and remove the container-- -f forces the stop then removes the
container.
docker rm -f <container_id_or_name>
(OR)-- Remove all stopped containers at once-- prune:- It removes things like
stopped containers, unused networks, dangling images, or unused volumes.
docker container prune
```

What is docker inspect?

- The `docker inspect` command is used to view detailed low-level information about:

- a container
- an image
- a volume
- a network, etc.
- It shows information in JSON format that Docker uses internally.

```
docker inspect <container_name_or_id>

docker inspect <image_name_or_id>
docker inspect <volume_name>
docker inspect <network_name>
```

 What kind of information does it show?

- For a container, it shows:

Info Type	Description
Config	The original config used to create the container
State	Is it running, paused, or exited
Mounts	Volumes attached
NetworkSettings	IP address, ports, bridge info
Env	Environment variables
Image	Image ID used to start container
Path, Args	The command used to start the container
RestartCount	How many times it's been restarted

Example:-

```
docker inspect my-container

[
  {
    "Id": "a1b2c3d4...",
    "State": {
      "Status": "running",
      "Running": true,
      "StartedAt": "2025-06-20T10:20:00Z"
    },
    "NetworkSettings": {
      "IPAddress": "172.17.0.2"
    },
    "Mounts": [
```

```
{
  "Type": "volume",
  "Name": "my-volume",
  "Destination": "/data"
}
]
```

🔑 Real-World Uses

Project Setup through docker

Build the Docker image file

```
docker build -t myfastapiapp .
```

Upload the image on docker hub

```
-- TO check login is correct
docker login
-- your-dockerhub-username : mohit2708
docker tag myfastapiapp <your-dockerhub-username>/myfastapiapp:latest
docker push <your-dockerhub-username>/myfastapiapp:latest
```

Docker troubleshooting commands

View Logs of a Container

```
docker log cont_id/cont_name
```

Detailed info about a container/image

```
docker inspect <container_or_image_name> # Detailed info about a container/image
```

Get Inside a Running Container (Debug)

```
docker exec -it <container_name> bash
-- Or use sh if bash is not available:
```



```
docker exec -it <container_name> sh
```

Clean Up Unused Stuff (Fix Space Issues)

```
docker system prune      # Remove stopped containers, networks, etc.
docker image prune        # Remove unused images
docker volume prune      # Remove unused volumes
```

Explain of -it

```
docker run -it ubuntu
```

- तो इसमें -it दो ऑप्शन्स (flags) का मेल है:
 - i (interactive):- इसका मतलब है कि कंटेनर का इनपुट (stdin) खुला रहेगा। यानी आप कंटेनर के अंदर टाइप कर सकेंगे, जैसे किसी कंप्यूटर पर कमांड टाइप करते हैं।
 - t (tty) इसका मतलब है कि एक टर्मिनल (terminal) बनाया जाएगा। ये टर्मिनल आपके लिए इंटरैक्टिव (interactive) सेशन जैसा अनुभव देगा, मतलब स्क्रीन पर आउटपुट साफ़ दिखेगा और आप कमांड लिख सकेंगे।

क्यों use करते हैं -it?

- अगर आप सिर्फ `docker run ubuntu` लिखेंगे, तो कंटेनर चलेगा, लेकिन आप उसके अंदर सीधे कमांड टाइप नहीं कर पाएंगे। कंटेनर के अंदर इंटरैक्टिवली काम करने के लिए आपको -it देना पड़ता है ताकि आप कंटेनर में एक टर्मिनल खोलकर कमांड चला सकें।
- मतलब:** -it से आप कंटेनर के अंदर सीधे बैठकर कमांड चला सकते हैं, जैसे कि आप अपने कंप्यूटर पर टर्मिनल में काम कर रहे हों।

उदाहरण 1: docker run ubuntu (बिना -it के)

- ये कमांड Ubuntu कंटेनर को रन करेगा, लेकिन आपको उसके अंदर इंटरैक्टिव शेल (**terminal**) नहीं मिलेगा।
- कंटेनर बिना टर्मिनल के चलता है और तुरंत बंद हो सकता है क्योंकि कोई कमांड नहीं दिया गया है जो कंटेनर को चलाए रखे।
- इसलिए आप कंटेनर के अंदर टाइप करके काम नहीं कर पाएंगे।

उदाहरण 2: docker run -it ubuntu

- ये कंटेनर को स्टार्ट करता है और एक टर्मिनल खोल देता है।
- आप कंटेनर के अंदर सीधे कमांड टाइप कर सकते हैं, जैसे कि `bash` शेल।
- उदाहरण के लिए, आप `ls`, `pwd`, या `apt update` जैसे कमांड चला सकते हैं।
- कंटेनर तब तक चलता रहेगा जब तक आप `exit` नहीं करते।

What is a Docker volume?

- Docker Volume is a special folder that is outside the Docker container but is associated with the container.
- It is used to keep the container's data safe, so that the data is not lost even when the container is deleted or recreated.
- (Docker Volume एक स्पेशल फोल्डर होता है जो Docker कंटेनर के बाहर होता है लेकिन कंटेनर के साथ जुड़ा रहता है। इसका इस्तेमाल कंटेनर के डेटा को सुरक्षित रखने के लिए किया जाता है, ताकि जब कंटेनर डिलीट या रीक्रिएट हो जाए तब भी डेटा खो ना जाए।)

Why use Docker volumes?

- Data persistence: When a container is removed, its writable layer is deleted too, so data stored inside the container disappears. Volumes keep the data intact.
- Sharing data: Volumes let you share data between multiple containers.
- Performance: Volumes are generally more efficient than storing data inside the container's filesystem.
- Backup and migration: Volumes can be backed up or migrated more easily than container data.
- डेटा सुरक्षित रखना: अगर आप कोई कंटेनर बंद या हटा देते हो, तो कंटेनर के अंदर जो भी डेटा होता है, वो चला जाता है। लेकिन अगर वो डेटा volume में रखा हो, तो वो सुरक्षित रहता है।
- डेटा शेयरिंग: एक volume को एक से ज्यादा कंटेनर में शेयर किया जा सकता है।
- बैकअप और ट्रांसफर करना आसान: Volume का डेटा होस्ट मशीन पर होता है, इसलिए इसका बैकअप लेना या दूसरे सिस्टम पर ट्रांसफर करना आसान होता है।
- फास्ट और सुरक्षित: Docker volume, कंटेनर के फाइल सिस्टम के मुकाबले ज्यादा तेज और सुरक्षित होता है।

Create a volume

```
docker volume create <volume_name>
```

List all Volumes

```
docker volume ls
```

Delete a Named volume

```
docker volume rm <volume_name>

# Remove unused local volumes
docker volume prune //for anonymous volumes
```

Mount Named volume with running container

```
docker run - -volume <volume_name>:<mount_path>
//or using - -mount
```

```
docker run - -mount type=volume,src=<volume_name>,dest=<mount_path>
```

Mount Anonymous volume with running container

```
docker run - -volume <mount_path>
```

To create a Bind Mount

```
docker run - -volume <host_path>:<container_path>  
//or using - -mount  
docker run - -mount type=bind,src=<host_path>,dest=<container_path>
```

what is Network?

List all networks

```
docker network ls
```

Create a network

```
docker network create <network_name>
```

Remove a network

```
docker network rm <network_name>
```

Remove all unused networks

```
docker network prune
```

Docker compose

```
docker compose -f file_name.yaml up -d  
docker compose -f file_name.yaml down
```

Dockerizing our app

```
FROM      # project run karne ke liye s/w requirements like node, pyhon
WORKDIR
COPY
RUN
CMD
EXPOSE
ENV
```

Publising Images

```
docker build -t devapnacollege/testapp # image build

# Login docker
docker login -u <username>

docker push devapnacollege/testapp # push in docker
```